

Snowflake Architecture & Cache with Practical Examples

1. Explain in your own words the three main layers of Snowflake architecture.

Practical:

Draw a simple diagram showing Storage, Compute (Virtual Warehouse), and Cloud Services . Explain what happens when you run a SELECT query.

Answer:

Storage Layer , Compute Layer , Cloud Services layer:

Cloud Services layer:

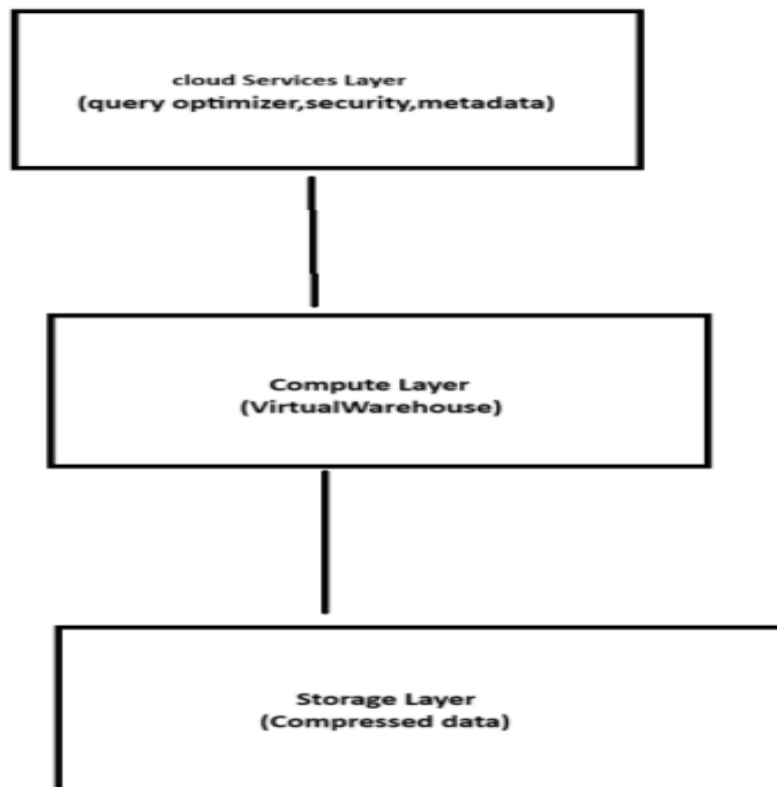
- Brain of Snowflake . Manages all coordination and metadata

Compute Layer (Virtual Warehouse):

- **Virtual Warehouse = a cluster of compute resources** . Executes SQL queries.

Storage Layer :

- Stores all data (tables, views, files) in **compressed, encrypte micro-partitions**.Uses cloud storage (AWS S3 / Azure Blob / GCP).



snowflake Architecture

What Happens When You Run a **SELECT** Query?

```
SELECT * FROM EMPLOYEE;
```

- Snowflake separates storage, compute, and services so data is stored efficiently, queries run independently, and system management is fully automated.

2. What is the role of the Storage layer in Snowflake?

Practical:

Create a table and insert some data. Show how data remains stored even if the warehouse is suspended.

Answer:

First I created one table and also i insert some data:

```
1 CREATE TABLE employee (  
2   id INT,  
3   name STRING,  
4   salary NUMBER  
5 );  
6  
7 INSERT INTO employee VALUES  
8 (1, 'Ravi', 50000),  
9 (2, 'Anita', 60000),  
10 (3, 'Kumar', 70000);  
11  
12 SELECT * FROM employee;
```

Results (just now)

Table	Chart	3 rows	63ms	Download	Refresh
ID	NAME	SALARY			
1	Ravi	50000			
2	Anita	60000			
3	Kumar	70000			

After i suspend a warehouse:

```
14 ALTER WAREHOUSE COMPUTE_WH SUSPEND;  
15
```

Results (just now)

Table	Chart	1 row	42ms	Download	Refresh
status					
1	Statement executed successfully.				

And also i resume and select the data in employee table:

```
22 ALTER WAREHOUSE COMPUTE_WH RESUME;  
23
```

Results (just now)

Table	Chart	1 row	65ms	Download	Refresh
status					
1	Statement executed successfully.				

```

22 ALTER WAREHOUSE COMPUTE_WH RESUME;
23
24 SELECT * FROM employee;
25
26
27
28
29

```

Results (just now)

Table Chart 3 rows 32ms

#	ID	NAME	SALARY
1	1	Ravi	50000
2	2	Anita	60000
3	3	Kumar	70000

- Data is still available. This proves storage is independent of compute.

The Storage Layer in Snowflake securely stores compressed and encrypted data and remains available even when the Virtual Warehouse is suspended.

3. Describe what a Virtual Warehouse is in Snowflake.

Practical:

Create a warehouse, run a query, then suspend the warehouse. Show how warehouse size affects query speed by resizing it.

Answer:

A **Virtual Warehouse** in Snowflake is a **compute engine** used to **execute SQL queries**.

Create a warehouse:

```

27 CREATE OR REPLACE WAREHOUSE demo_wh
28 WITH
29 WAREHOUSE_SIZE = 'XSMALL'
30 AUTO_SUSPEND = 60
31 AUTO_RESUME = TRUE;

```

Results (just now)

Table Chart 1 row 119ms

#	status
1	Warehouse DEMO_WH successfully created.

Run a query:

```

35 SELECT COUNT(*) FROM employee;
36

```

Results (just now)

Table Chart 1 row 57ms

#	COUNT(*)
1	3

suspend the warehouse:

```

36 ALTER WAREHOUSE demo_wh SUSPEND;
37
38

```

Results (just now)

Table Chart 1 row 50ms

#	status
1	Statement executed successfully.

Resize Warehouse to Improve Query Speed:

```
39 ALTER WAREHOUSE demo_wh SET WAREHOUSE_SIZE = 'LARGE';
```

Results (just now)	
Table	Chart
1 row	63ms
1	Statement executed successfully.

Run the Same Query Again:

```
41 SELECT COUNT(*) FROM employee;
```

Results (just now)	
Table	Chart
1 row	28ms
1	3

More size = more compute = faster queries

- A Virtual Warehouse in Snowflake is an independent compute cluster used to execute SQL queries and can be scaled or suspended without affecting stored data.

4. Explain micro-partitions and their importance.

Practical:

Run a query with a filter (e.g., WHERE AMOUNT > 1000) and explain how Snowflake reads only relevant micro-partitions to speed up the query.

Answer:

Micro-partitions are **small, automatically created chunks of data** in Snowflake's **Storage Layer**.

Practical Demonstration - create table and insert data:

```
44 CREATE OR REPLACE TABLE orders (  
45     order_id INT,  
46     customer STRING,  
47     amount NUMBER  
48 );  
49  
50 INSERT INTO orders VALUES  
51 (1, 'A', 500),  
52 (2, 'B', 1200),  
53 (3, 'C', 3000),  
54 (4, 'D', 800),  
55 (5, 'E', 4500);
```

Results (just now)	
Table	Chart
1 row	515ms
1	5

Run a Filtered Query:

```
57 | SELECT *
58 | FROM orders
59 | WHERE amount > 1000;
60 |
```

Results (just now)

Table Chart 3 rows 312ms

	ORDER_ID	CUSTOMER	AMOUNT
1	2	B	1200
2	3	C	3000
3	5	E	4500

What Happens Internally:

- Data is already stored in **micro-partitions**
- Each micro-partition has metadata like:
 - MIN(amount)
 - MAX(amount)

Micro-partitions are small, automatically managed data blocks that allow Snowflake to scan only relevant data using metadata, significantly improving query performance.

5. What is the Cloud Services layer?

Practical:

Show a login process and role-based access in Snowflake. Explain how Cloud Services verify permissions before query execution.

Answer:

The Cloud Services Layer is the **control and management layer of Snowflake**. It acts as the **brain that manages security, metadata, and query coordination**.

First we check the user and role in current manner:

```
63 | SELECT CURRENT_USER(), CURRENT_ROLE();
```

Results (just now)

Table Chart 1 row 36ms

	CURRENT_USER()	CURRENT_ROLE()
1	ROSHINIRL	ANALYST_ROLE

After creating one new role and grand the permission:

```
65 | CREATE OR REPLACE ROLE analyst_role;
66 |
67 | GRANT USAGE ON DATABASE my_db TO ROLE analyst_role;
68 | GRANT USAGE ON SCHEMA my_db.public TO ROLE analyst_role;
69 | GRANT SELECT ON TABLE orders TO ROLE analyst_role;
70 |
```

Results (just now)

Table Chart 1 row 64ms

	status
1	Statement executed successfully.

After we assign role to user and switch role:

```
85
86 GRANT ROLE analyst_role TO USER roshinirl;
87
88 USE ROLE analyst_role;
```

Results (just now)

Table Chart 1 row 25ms

#	status
1	Statement executed successfully.

Run a Query:

```
90 SELECT * FROM orders;
91
```

Results (just now)

Table Chart 5 rows 40ms

#	ORDER_ID	CUSTOMER	AMOUNT
1	1	A	500
2	2	B	1200
3	3	C	3000
4	4	D	800
5	5	E	4500

- The Cloud Services layer manages authentication, authorization, metadata, and query optimization, ensuring secure and efficient query execution in Snowflake.

6. Name and describe the three types of cache Snowflake uses.

Practical:

Run the same query twice and measure execution time to demonstrate Result Cache.
Restart warehouse and rerun query to explain Local Disk Cache.

Answer:

Snowflake uses **three levels of caching** to improve query performance and reduce cost.

6.1 Result Cache

- Stores the **final result set** of a query.
- Cached for **24 hours**.
- No warehouse is used → **zero compute cost**.

Run query first time - Execution time: **Longer** (data read + processing)

```
92 | SELECT * FROM orders WHERE AMOUNT > 1000;
93
94
```

Results (just now)

Table Chart

3 rows 33ms

ORDER_ID	CUSTOMER	AMOUNT
1	B	1200
2	C	3000
3	E	4500

Run same query again - Execution time: **Very fast (milliseconds)**

```
92 | SELECT * FROM orders WHERE AMOUNT > 1000;
93
94
```

Results (just now)

Table Chart

3 rows 23ms

ORDER_ID	CUSTOMER	AMOUNT
1	B	1200
2	C	3000
3	E	4500

6.2 Local Disk Cache (Warehouse Cache)

- Data cached on the **local SSD disks** of a virtual warehouse.
- Stores micro-partition data read during previous queries.

Disable Result Cache - **Run query in this Execution time is Moderate**

```
94 | ALTER SESSION SET USE_CACHED_RESULT = FALSE;
95
96 | SELECT * FROM orders WHERE AMOUNT > 1000;
97
```

Results (just now)

Table Chart

3 rows 1.3s

ORDER_ID	CUSTOMER	AMOUNT
1	B	1200
2	C	3000
3	E	4500

Run again - **Faster than first run**

```
96 | SELECT * FROM orders WHERE AMOUNT > 1000;
97
98
```

Results (1 minute ago)

Table Chart

3 rows 70ms

ORDER_ID	CUSTOMER	AMOUNT
1	B	1200
2	C	3000
3	E	4500

6.3 Remote Disk Cache (Cloud Storage Cache)

- Data stored in Snowflake's **central cloud storage** (S3/Azure Blob/GCS).
- Always available.

The warehouse will restart in clear local cache.

```

98
99 ALTER WAREHOUSE COMPUTE_WH SUSPEND;
100 ALTER WAREHOUSE COMPUTE_WH RESUME;
101

```

Results (just now)

Table Chart 1 row 45ms

#	status
1	Statement executed successfully.

*Run query again - Execution time: **Slower again***

```

95
96 SELECT * FROM orders WHERE AMOUNT > 1000;
97

```

Results (just now)

Table Chart 3 rows 448ms

#	ORDER_ID	CUSTOMER	AMOUNT
1	2	B	1200
2	3	C	3000
3	5	E	4500

- Local Disk Cache is cleared when warehouse restarts.
- Snowflake now reads from Remote Disk Cache (Cloud Storage).

7. Explain Result Cache.

Practical:

Run a COUNT(*) query twice and note that the second run doesn't consume compute resources

Answer:

*Result Cache stores the **final output of a query** in the **Cloud Services layer** of Snowflake.*

- Cached for **24 hours**
- **No virtual warehouse is used**
- Returned instantly if the same query is executed again

*Run Query First Time - Execution time: **Slower***

```
102 | SELECT COUNT(*) FROM orders;
103
```

Results (just now)	
Table	Chart
# COUNT(*)	
1	5

Run Same Query Again - Execution time : **Milliseconds**

```
102 | SELECT COUNT(*) FROM orders;
103
```

Results (just now)	
Table	Chart
# COUNT(*)	
1	5

- Result Cache stores the final result of a query for 24 hours and returns it without using compute when the same query is rerun.

8. How does Local Disk Cache differ from Result Cache?

Practical:

Run a query, suspend warehouse, restart it, run the query again, and observe performance change

Answer:

- Result Cache : **Stores the final query result**, not raw data.
- Local Disk Cache : Stores **micro-partitions (raw data)** on the **warehouse's local SSD**

Run a query (first time) - It is slower, Reads from cloud storage

```
101 | -----
102 | SELECT COUNT(*) FROM orders;
103
```

Results (just now)	
Table	Chart
# COUNT(*)	
1	5

Run the SAME query again - Execution time: milliseconds (Result Cache)

```
102 | SELECT COUNT(*) FROM orders;
```

results (just now)	
Table	Chart
# COUNT(*)	
1	5

After Suspend Warehouse - Resume Warehouse

```
98 |
99 | ALTER WAREHOUSE COMPUTE_WH SUSPEND;
100 | ALTER WAREHOUSE COMPUTE_WH RESUME;
```

Results (just now)	
Table	Chart
status	
1	Statement executed successfully.

Run again the query - Faster than first run. Slower than result cache

```
102 | SELECT COUNT(*) FROM orders;
```

results (just now)	
Table	Chart
# COUNT(*)	
1	5

Result Cache stores query results and works even when W.H is stopped, while Local Disk Cache stores data in the W.H memory and is cleared when the warehouse is suspended.

9. What happens to caches when table data changes?

Practical:

Run a query twice, insert new rows, run again and explain why the Result Cache is invalidated and query runs fresh.

Answer:

When table data changes, the Result Cache is invalidated automatically, because Snowflake must return fresh and correct data.

Run Query First Time - Reads from cloud storage

```
41 | SELECT COUNT(*) FROM employee;
```

results (just now)	
Table	Chart
# COUNT(*)	
1	3

Run Same Query Again - Execution time: milliseconds

```
41 | SELECT COUNT(*) FROM employee;
```

#	COUNT(*)
1	3

*Insert New Data - This modifies table data. Run Same Query **Again***

```
106 | INSERT INTO orders VALUES (101, 'Laptop', 45000);
107 |
108 | SELECT * FROM orders;
```

#	ORDER_ID	CUSTOMER	AMOUNT
1	1	A	500
2	2	B	1200
3	3	C	3000
4	4	D	800
5	5	E	4500
6	101	Laptop	45000

If data changes → result cache breaks

When table data changes, Snowflake invalidates the Result Cache to ensure query results are **always accurate**, so the query is executed again using fresh data.

10. Describe the difference between first and second run of the same query.

Practical:

Run a query once (full compute), then immediately run again (Result Cache), and explain the difference in resource usage.

Answer:

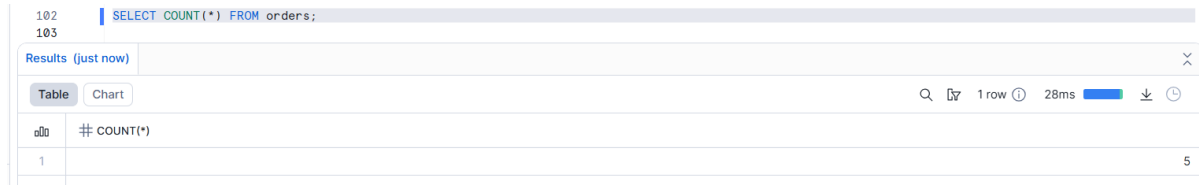
First run uses compute and scans data. **Second run** uses Result Cache (no compute).

Run the query (First Time) - Work is done

```
102 | SELECT COUNT(*) FROM orders;
```

#	COUNT(*)
1	5

Run the SAME query again immediately - Answer reused



The screenshot shows a Snowflake query interface. The query bar contains `SELECT COUNT(*) FROM orders;`. Below the query bar, the 'Results (just now)' tab is active. It shows a table with one column labeled `# COUNT(*)` and one row with the value `5`. The interface also shows '1 row' and '28ms' execution time.

COUNT(*)
5

In the **first run**, Snowflake scans data and uses compute, while in the **second run**, it returns the result from cache without using the warehouse, making it much **faster and cost-efficient**.

11. How does Snowflake's compute scaling work?

Practical:

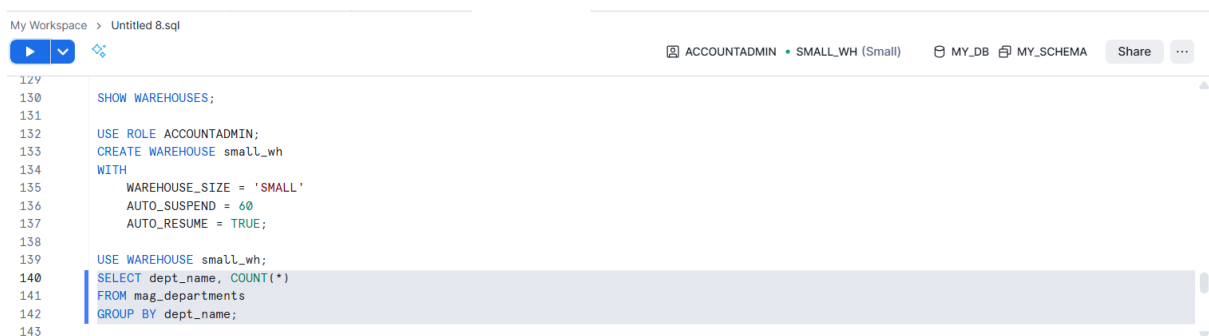
Run a complex query on a small warehouse, then increase warehouse size and rerun the query, observing improved speed.

Answer:

Snowflake compute scaling means **increasing or decreasing warehouse size** to control query performance.

- **Bigger warehouse** → Faster queries
- **Smaller warehouse** → Lower cost
- Data storage is not affected

Run Query on Small Warehouse and also run a complex query



The screenshot shows a Snowflake SQL editor with a query that creates a small warehouse and runs a complex query. The query is as follows:

```
129
130 SHOW WAREHOUSES;
131
132 USE ROLE ACCOUNTADMIN;
133 CREATE WAREHOUSE small_wh
134 WITH
135     WAREHOUSE_SIZE = 'SMALL'
136     AUTO_SUSPEND = 60
137     AUTO_RESUME = TRUE;
138
139 USE WAREHOUSE small_wh;
140 SELECT dept_name, COUNT(*)
141 FROM mag_departments
142 GROUP BY dept_name;
143
```

*Query runs **slow** - Limited CPU & memory - More execution time*

Increase Warehouse Size

The screenshot shows the Snowflake SQL Editor interface. The query editor contains the following SQL code:

```
132 USE ROLE ACCOUNTADMIN;  
133 CREATE WAREHOUSE small_wh  
134 WITH  
135     WAREHOUSE_SIZE = 'SMALL',  
136     AUTO_SUSPEND = 60,  
137     AUTO_RESUME = TRUE;  
138  
139 USE WAREHOUSE small_wh;  
140 SELECT dept_name, COUNT(*)  
141 FROM mag_departments  
142 GROUP BY dept_name;  
143  
144  
145 ALTER WAREHOUSE small_wh  
146 SET WAREHOUSE_SIZE = MEDIUM;
```

Below the query editor, the 'Results (just now)' section shows a table with one row: 'Statement executed successfully.'.

A dropdown menu is open, showing the 'Use warehouse...' list. The selected warehouse is 'SMALL_WH' (Medium). The list includes:

- SMALL_WH (Medium) ✓
- COMPUTE_WH (X-Small)
- DEMO_WH (Large)
- MY_WAREHOUSE (X-Small)
- MY_WH (X-Small)
- SNOWFLAKE_LEARNING_WH (X-Small)
- SYSTEM\$STREAMLIT_NOTEBOOK_WH (X-Small)

Run the Same Query Again

The screenshot shows the Snowflake SQL Editor interface. The query editor contains the following SQL code:

```
147 SELECT dept_name, COUNT(*)  
148 FROM mag_departments  
149 GROUP BY dept_name;
```

Below the query editor, the 'results (just now)' section shows a table with 5 rows:

	DEPT_NAME	COUNT(*)
1	Sales	1
2	Operations	1
3	IT	1
4	HR	1
5	Finance	1

Query runs fast - More parallel process - Same data, same query - Higher compute usage

12. Explain external stage vs internal stage.

Practical:

Create an internal stage and upload a file. Create an external stage pointing to S3 and explain the differences.

Answer:

- **Internal Stage** → Storage managed **inside Snowflake**
- **External Stage** → Storage located **outside Snowflake** (S3, Azure, GCS)

Create an Internal Stage & Upload File - I upload directly into snowflake

```

152 CREATE STAGE my_internal_stage;
153
154 SELECT * FROM "MY_DB"."MY_SCHEMA"."FINANCIAL"
155
156
157
158

```

Results (just now)

Table Chart 6 rows 26ms

	C1	C2	C3	C4
1	null	null	null	null
2	(101	'Arjun'	25000)	null
3	(102	'Riya'	18000)	null
4	(103	'Karthik'	32000)	null
5	(104	'Meena'	21000)	null
6	(105	'Rahul'	40000);	null

Create an external stage pointing to S3 bucket

```

1 CREATE OR REPLACE STAGE ext_stage
2 URL='s3://snowflake-ingestion-new'
3 CREDENTIALS=(
4   AWS_KEY_ID='AKIAUSXDYNKBWY4IAZVD',
5   AWS_SECRET_KEY='ievsPJeNLZHFR+zqiE/IyLtc70VMtC+wRiDWER+x'
6 );
7
8 ALTER STAGE ext_stage
9 SET FILE_FORMAT = (TYPE = 'XML');
10
11 LIST @ext_stage;
12

```

Results (just now)

Table Chart 2 rows 194ms

	name	size	md5	last_modified
1	s3://snowflake-ingestion-new/employees.xml	1111	fdff4ceebd11059e52a5d2f2273e1ac4	Fri, 26 Dec 2025 10:51:12 GMT
2	s3://snowflake-ingestion-new/financial_data.csv	123	b6fd44c76209b65da2a7abfd33ca927f	Tue, 30 Dec 2025 05:19:13 GMT

13. Explain the statement:

“Data never moves, compute comes and goes.”

Practical:

Show data stored in table remains the same regardless of warehouse start/stop and resizing.

Answer:

- In Snowflake, data is stored separately from compute.
- Compute (warehouse) can be started, stopped, or resized without affecting data.
- Data = Permanent ; Compute = Temporary

Create a Table and Insert Data - Check data

```

157 CREATE OR REPLACE TABLE sales_demo (
158     id INT,
159     product STRING,
160     amount INT
161 );
162
163 INSERT INTO sales_demo VALUES
164 (1, 'Laptop', 50000),
165 (2, 'Mobile', 30000);
166
167 SELECT * FROM sales_demo;
168

```

Results (just now)

Table Chart 2 rows 260ms

# ID	PRODUCT	AMOUNT
1	Laptop	50000
2	Mobile	30000

Suspend - Compute stopped , Data still stored safely and Resume the warehouse

```

169 ALTER WAREHOUSE small_wh SUSPEND;
170
171 ALTER WAREHOUSE small_wh RESUME;
172

```

Results (just now)

Table Chart 1 row 52ms

status
Statement executed successfully.

Run Query Again - Data is **still there**, Only compute restarted.

```

173 SELECT * FROM sales_demo;
174

```

Results (just now)

Table Chart 2 rows 61ms

# ID	PRODUCT	AMOUNT
1	Laptop	50000
2	Mobile	30000

Resize Warehouse , Run query again - Data unchanged , Query runs faster- Only compute increased

```

175 ALTER WAREHOUSE small_wh SET WAREHOUSE_SIZE = MEDIUM;
176
177 SELECT * FROM sales_demo;

```

Results (just now)

Table Chart 2 rows 32ms

# ID	PRODUCT	AMOUNT
1	Laptop	50000
2	Mobile	30000